

**LAPORAN RESMI MATA KULIAH WORKSHOP APLIKASI  
BERBASIS WEB LAPORAN CRUD WITH LARAVEL**



Nama : Muhammad Yoga Ananda Satria

NRP : 3124500036

Dosen Pengajar : Mu'arifin S.ST., M.T

**PROGRAM STUDI D3 TEKNIK INFORMATIKA  
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA (PENS)  
TAHUN 2025**

## A. Tujuan Pembelajaran

Tujuan dari praktikum ini adalah agar mahasiswa mampu memahami dan menguasai konsep dasar **migration** dan **model** dalam framework Laravel. Mahasiswa diharapkan dapat membuat tabel baru di dalam database menggunakan migration, serta mengimplementasikan model untuk menghubungkan tabel dengan logika aplikasi. Selain itu, mahasiswa juga dituntut untuk dapat melakukan operasi dasar **CRUD (Create, Read, Update, Delete)** secara lengkap melalui pemanfaatan migration, model, controller, dan view.

Praktikum ini juga bertujuan untuk membiasakan mahasiswa dalam menggunakan **Git** sebagai version control dalam setiap tahapan pengembangan aplikasi, mulai dari melakukan commit, push, hingga manajemen branch. Dengan demikian, mahasiswa memperoleh pemahaman menyeluruh tentang bagaimana konsep database, kode program, serta manajemen versi dapat diintegrasikan secara efektif dalam proses pengembangan aplikasi web berbasis Laravel.

## B. Dasar Teori

Migration adalah sebuah fitur penting dalam Laravel yang digunakan untuk mengelola struktur database secara terprogram. Dengan migration, pengembang tidak perlu membuat tabel secara manual melalui phpMyAdmin, melainkan dapat mendefinisikan tabel, kolom, maupun relasi antar tabel menggunakan kode PHP. Hal ini membuat perubahan database lebih terstruktur, terdokumentasi, dan mudah dilacak.

Sementara itu, **model** dalam Laravel merupakan representasi tabel pada database yang digunakan untuk melakukan interaksi data. Model memanfaatkan **Eloquent ORM (Object Relational Mapping)** yang memungkinkan pengembang mengeksekusi query database menggunakan objek dan method, sehingga lebih efisien dan mudah

dipahami. Konsep **CRUD (Create, Read, Update, Delete)** yang menjadi dasar pengolahan data juga sangat erat kaitannya dengan penggunaan migration dan model. CRUD diimplementasikan melalui kombinasi controller, model, dan view yang saling terhubung dalam pola arsitektur **MVC (Model-View-Controller)**. Selain itu, penggunaan **version control** seperti Git menjadi hal yang tidak kalah penting karena membantu pengembang untuk mengelola perubahan kode, melakukan kolaborasi tim, serta menjaga histori pengembangan aplikasi agar lebih sistematis dan profesional.

## C. Jawaban

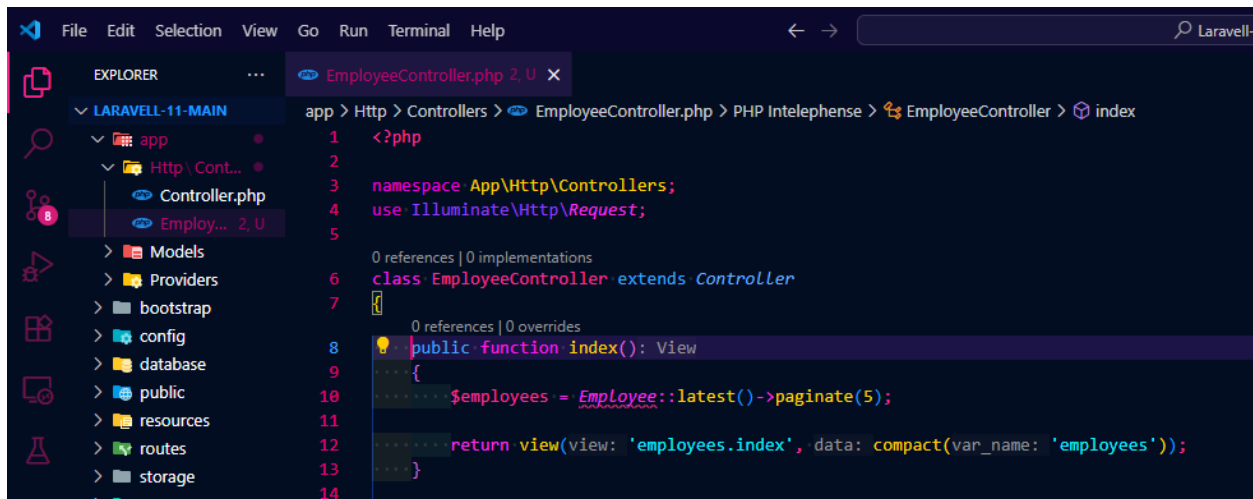
### #Menampilkan Data

1. Membuat Controller *Employee* dengan perintah `php artisan make:controller EmployeeController --resource`, Kita harus menjalankan sebuah perintah yang dimana perintah tersebut akan membuat kita sebuah file controller pada folder controller atau pada folder yang menampung semua controller yang sudah dibuat. Berikut output di dalam terminal saat menjalankan perintahnya:

```
PS C:\laragon\www\Laravell-11-main> php artisan make:controller EmployeeController --resource  
  
INFO Controller [C:\laragon\www\Laravell-11-main\app\Http\Controllers\EmployeeController.php] created successfully.  
  
PS C:\laragon\www\Laravell-11-main> |
```

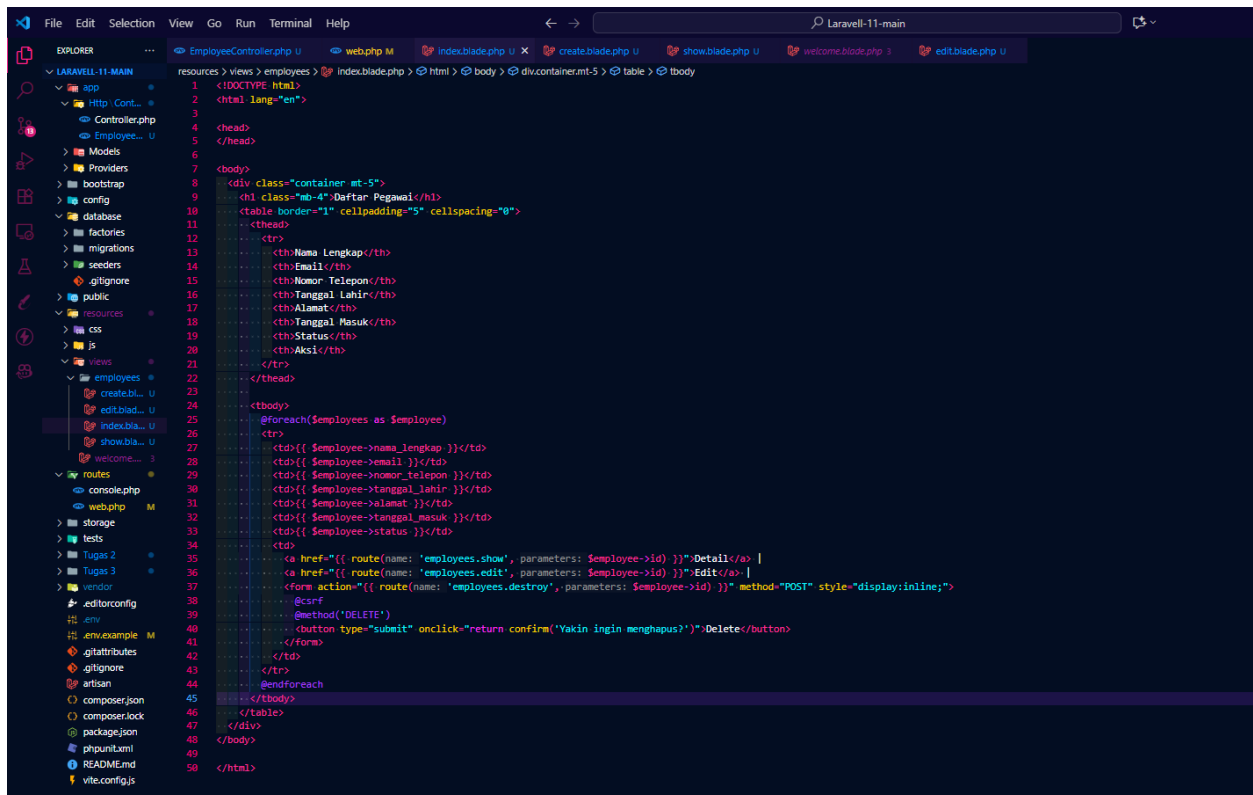
2. Membuka Controller `EmployeeController.php` dan memasukan kode untuk mengambil data dari database pada method `index`

Setelah di bukaa kita harus mengganti isi nyaa seperti pada outpud di bawah ini:



```
1 <?php
2
3 namespace App\Http\Controllers;
4 use Illuminate\Http\Request;
5
6 class EmployeeController extends Controller
7 {
8     //
9     public function index(): View
10     {
11         $employees = Employee::latest()->paginate(5);
12         return view('employees.index', data: compact('employees'));
13     }
14 }
```

3. Membuka folder resources => views setelah itu membuat folder baru yang bernama employees, dan membuat file dengan nama index.blade.php, setelah itu memasukkan code yang sudah diberikan di file, dimana file tersebut adalah file awal dari halaman untuk melihat hasil dari data yang sudah di push ke database,



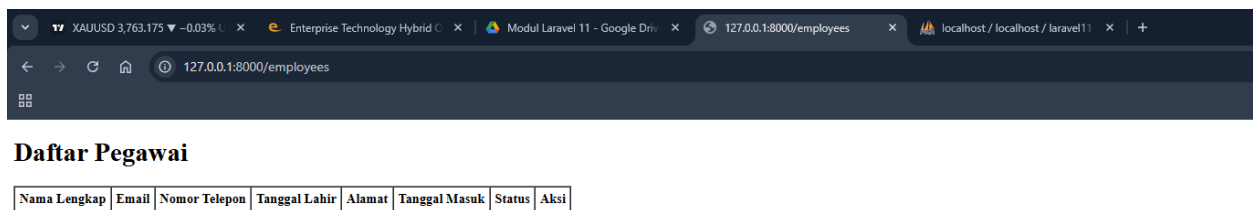
```
1 <doctype html>
2 <html lang="en">
3 <head>
4 </head>
5 <body>
6 <div class="container mt-5">
7 <h1 class="mb-4">Daftar Pegawai</h1>
8 <table border="1" cellpadding="5" cellspacing="0">
9 <thead>
10 <tr>
11 <th>Nama Lengkap</th>
12 <th>Email</th>
13 <th>Nomor Telepon</th>
14 <th>Tanggal Lahir</th>
15 <th>Alamat</th>
16 <th>Tanggal Masuk</th>
17 <th>Status</th>
18 <th>Aksi</th>
19 </tr>
20 </thead>
21 <tbody>
22 <@foreach($employees as $employee)>
23 <tr>
24 <td>{{ $employee->nama_lengkap }}</td>
25 <td>{{ $employee->email }}</td>
26 <td>{{ $employee->nomor_telepon }}</td>
27 <td>{{ $employee->tanggal_lahir }}</td>
28 <td>{{ $employee->alamat }}</td>
29 <td>{{ $employee->tanggal_masuk }}</td>
30 <td>{{ $employee->status }}</td>
31 <td>
32 <a href="{{ route('employees.show', parameters: $employee->id) }}">Detail</a> |
33 <a href="{{ route('employees.edit', parameters: $employee->id) }}">Edit</a> |
34 <a href="{{ route('employees.destroy', parameters: $employee->id) }}" method="POST" style="display:inline;">Delete</a>
35 </td>
36 </tr>
37 </tbody>
38 </table>
39 </div>
40 </body>
41 </html>
```

4.menambahkan route pada routes/web.php dengan menggunakan resources agar tidak perlu membuat route satu persatu,import controllers dengan menggunakan code use/App/Http/Controllers/EmployeeController.Berikut tampilannn nyaa pada VSCode.



```
routes > web.php
1  <?php
2
3  use Illuminate\Support\Facades\Route;
4  use App\Http\Controllers\EmployeeController;
5
6  Route::resource(name: 'employees', controller: EmployeeController::class);
```

5.setelah memasukan code tersebut jalankan pada browser <http://127.0.0.1:8000/employees>,dan akan memunculkan output di bawan ini



## #INSERT DATA

1.Langkah pertama untuk memasukan data kedalam tabel dalam database yang perlu di lakukaknn adalah dengan menambahkan kode pada method **create**(menampilkan from insert) dan store (menambahkan kode untuk insert data ke tabel).seperti gambar di bawah ini langkah awal nya yaitu membuka file EmployeeController.php dan memasukkan code seperti gambar di bawah ini pada method **create** dan juga **store**:

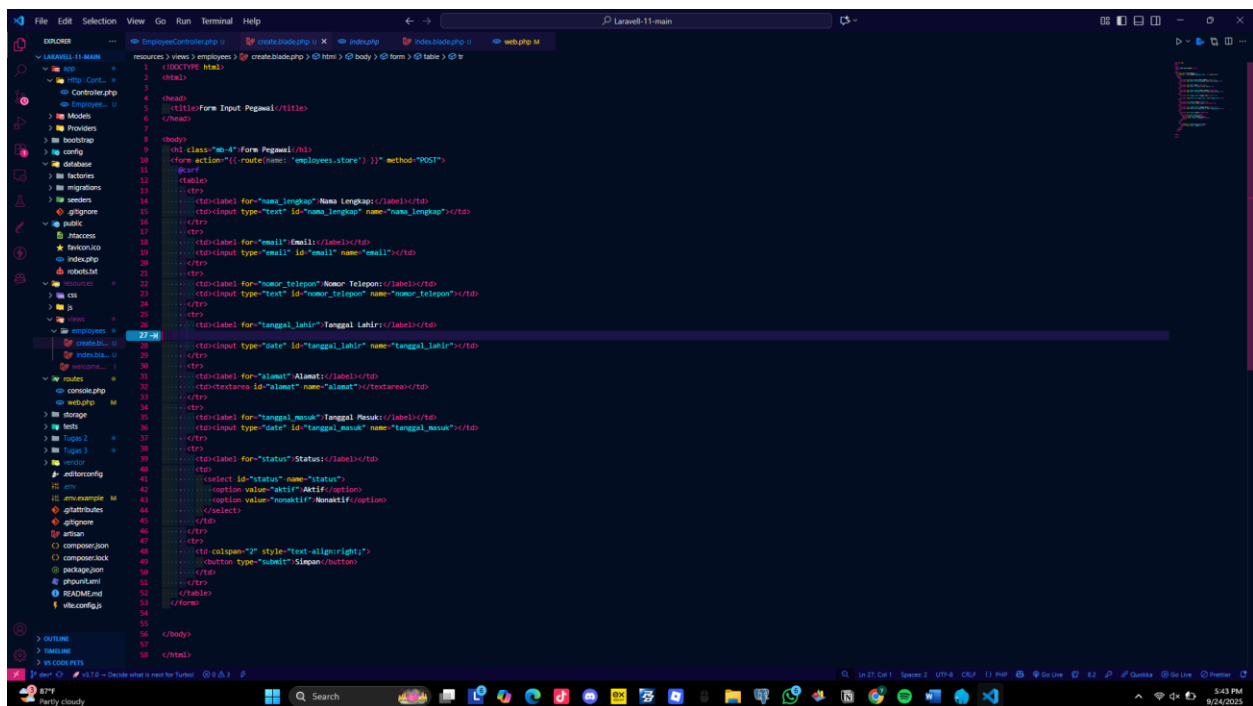
```

... public function create(): mixed
... {
...     return view(view: 'employees.create');
... }

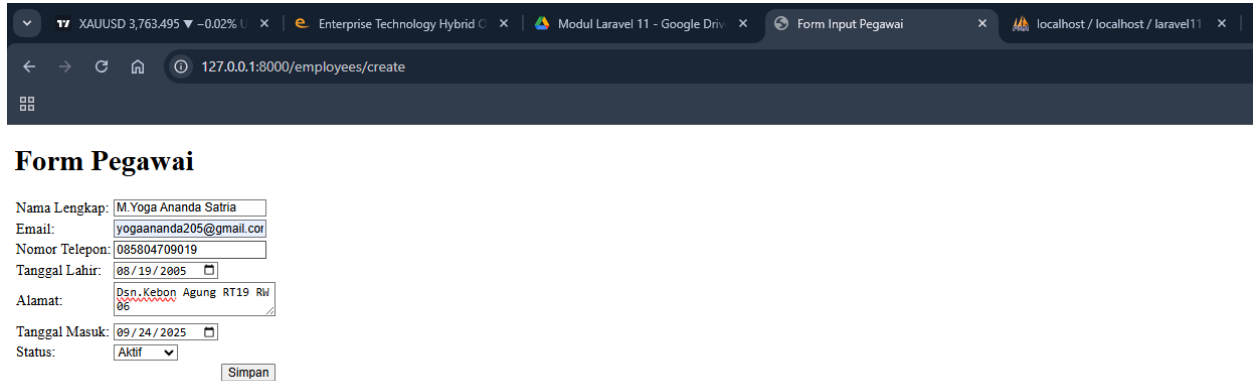
... /**
...  * Store a newly created resource in storage.
...  */
... 0 references | 0 overrides
... public function store(Request $request): mixed
... {
...     $request->validate(rules: [
...         'nama_lengkap' => 'required|string|max:255',
...         'email' => 'required|email|max:255',
...         'nomor_telepon' => 'required|string|max:20',
...         'tanggal_lahir' => 'required|date',
...         'alamat' => 'required|string|max:255',
...         'tanggal_masuk' => 'required|date',
...         'status' => 'required|string|max:50',
...     ]);
...     Employee::create(attributes: $request->all());
...     return redirect()->route('employees.index');
... }

```

## 2. Membuat file create.blade.php dalam direktori employees untuk membuat Form input data



### 3. Contoh Tampilan Output dari code di atas



**Form Pegawai**

Nama Lengkap:

Email:

Nomor Telepon:

Tanggal Lahir:

Alamat:

Tanggal Masuk:

Status:

4. Klik Simpan jika berhasil maka akan menampilkan tampilan seperti berikut



**Daftar Pegawai**

| Nama Lengkap         | Email                   | Nomor Telepon | Tanggal Lahir | Alamat                     | Tanggal Masuk | Status | Aksi                                                               |
|----------------------|-------------------------|---------------|---------------|----------------------------|---------------|--------|--------------------------------------------------------------------|
| M.Yoga Ananda Satria | yogaananda205@gmail.com | 085804709019  | 2005-08-19    | Dsn.Kebon Agung RT19 RW 06 | 2025-09-24    | aktif  | <a href="#">Detail</a> <a href="#">Edit</a> <a href="#">Delete</a> |

## #MENAMPILKAN DATA

1.detail untuk menampilkan data bisa kita lihat pada method show,langkah awal yaitu membuka file EmployeeController.php lalu menampahkan code seperti gambar di bawah ini:

```
public function show(string $id): View  
{  
    $employee = Employee::find(id: $id);  
    return view(view: 'employees.show', data: compact(var_name: 'employee'));  
}
```

2.Selanjutnya setelahh memasukan code di atas kita membuat file baru dengan nama show.blade.php pada direktori employees,lalu masukkan code berikut agar kita bisa menampilkan data secara detail:

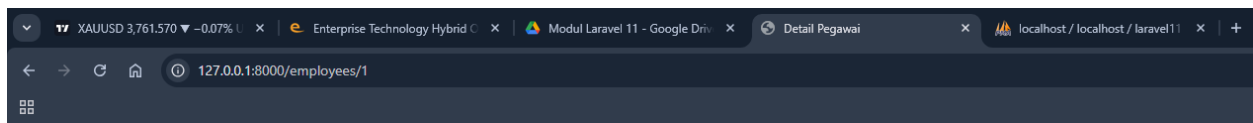
```

sources / views / employees / show.blade.php / html / body /
1  <!DOCTYPE html>
2  <html>
3
4  <head>
5  <title>Detail Pegawai</title>
6  </head>
7
8  <body>
9  <h1>Detail Pegawai</h1>
10 <table border="1" cellpadding="8" cellspacing="0">
11 <tr>
12 <th>Nama Lengkap</th>
13 <td>{{ $employee->nama_lengkap }}</td>
14 </tr>
15 <tr>
16 <th>Email</th>
17 <td>{{ $employee->email }}</td>
18 </tr>
19 <tr>
20 <th>Nomor Telepon</th>
21 <td>{{ $employee->nomor_telepon }}</td>
22 </tr>
23 <tr>
24 <th>Tanggal Lahir</th>
25 <td>{{ $employee->tanggal_lahir }}</td>
26 </tr>
27 <tr>
28 <th>Alamat</th>
29 <td>{{ $employee->alamat }}</td>
30 </tr>
31 <tr>
32 <th>Tanggal Masuk</th>
33 <td>{{ $employee->tanggal_masuk }}</td>
34 </tr>
35 <tr>
36 <th>Status</th>
37 <td>{{ $employee->status }}</td>
38 </tr>
39 </table>
40 </body>
41
42 </html>

```

3. setelah memasukkan code di atas uji coba di chrome untuk menampilkan data pada route <http://127.0.0.1:8000/employees>, maka akan muncul tabell seperti berikut:





## Detail Pegawai

|               |                            |
|---------------|----------------------------|
| Nama Lengkap  | M.Yoga Ananda Satria       |
| Email         | yogaananda205@gmail.com    |
| Nomor Telepon | 085804709019               |
| Tanggal Lahir | 2005-08-19                 |
| Alamat        | Dsn.Kebon Agung RT19 RW 06 |
| Tanggal Masuk | 2025-09-24                 |
| Status        | aktif                      |

## UPDATE DATA

1. Buka file **EmployeeController.php** dan tambahkan code seperti di bawah ini pada method **edit** dan juga **update**:

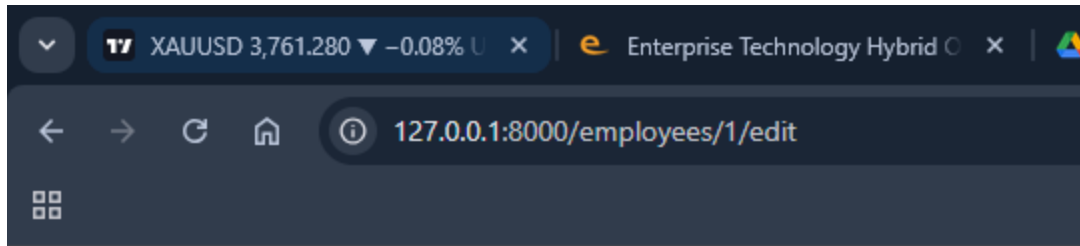
```
public function edit(string $id): View
{
    $employee = Employee::find(id: $id);
    return view(view: 'employees.edit', data: compact(var_name: 'employee'));
}
```

```
public function update(Request $request, string $id): RedirectResponse
{
    $request->validate(rules: [
        'nama_lengkap' => 'required|string|max:255',
        'email' => 'required|email|max:255',
        'nomor_telepon' => 'required|string|max:20',
        'tanggal_lahir' => 'required|date',
        'alamat' => 'required|string|max:255',
        'tanggal_masuk' => 'required|date',
        'status' => 'required|string|max:50',
    ]);
    $employee = Employee::findOrFail(id: $id);
    $employee->update(attributes: $request->only(keys: [
        'nama_lengkap',
        'email',
        'nomor_telepon',
        'tanggal_lahir',
        'alamat',
        'tanggal_masuk',
        'status',
    ]));
    return redirect()->route(route: 'employees.index');
}
```

2.Lanjut dengan membuat file edit.blade.php pada direktori employees untuk membuat **Form edit** seperti gambar berikut:

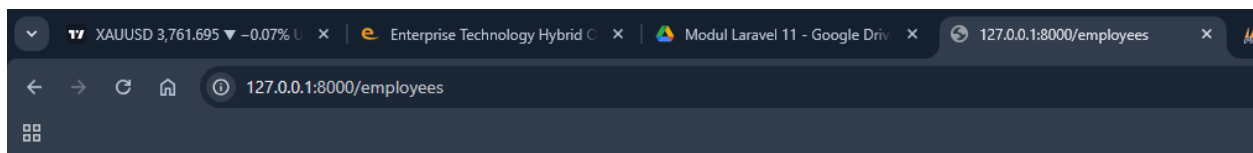
```
<h2>Edit Data Pegawai</h2>
<form action="{{ route('employees.update', parameters: $employee->id) }}" method="POST">
    @csrf
    @method('PUT')
    <table>
        <tr>
            <td>Nama Lengkap</td>
            <td><input type="text" name="nama_lengkap" value="{{ old(key: 'nama_lengkap', default: $employee->nama_lengkap) }}"></td>
        </tr>
        <tr>
            <td>Email</td>
            <td><input type="email" name="email" value="{{ old(key: 'email', default: $employee->email) }}"></td>
        </tr>
        <tr>
            <td>Nomor Telepon</td>
            <td><input type="text" name="nomor_telepon" value="{{ old(key: 'nomor_telepon', default: $employee->nomor_telepon) }}"></td>
        </tr>
        <tr>
            <td>Tanggal Lahir</td>
            <td><input type="date" name="tanggal_lahir" value="{{ old(key: 'tanggal_lahir', default: $employee->tanggal_lahir) }}"></td>
        </tr>
        <tr>
            <td>Alamat</td>
            <td><input type="text" name="alamat" value="{{ old(key: 'alamat', default: $employee->alamat) }}"></td>
        </tr>
        <tr>
            <td>Tanggal Masuk</td>
            <td><input type="date" name="tanggal_masuk" value="{{ old(key: 'tanggal_masuk', default: $employee->tanggal_masuk) }}"></td>
        </tr>
        <tr>
            <td>Status</td>
            <td>
                <select name="status">
                    <option value="aktif" {{ old(key: 'status', default: $employee->status) == 'aktif' ? 'selected' : '' }}>Aktif</option>
                    <option value="tidak aktif" {{ old(key: 'status', default: $employee->status) == 'tidak aktif' ? 'selected' : '' }}>Tidak
                        Aktif</option>
                </select>
            </td>
        </tr>
        <tr>
            <td colspan="2">
                <button type="submit">Update</button>
            </td>
        </tr>
    </table>
</form>
```

3.Uji coba dengan masuk ke halaman route <http://127.0.0.1:8000/employees> kemudian klik **Edit** pada bagian aksi jika berhasil maka tampilannya akan muncul seperti gambar di bawah ini



## Edit Data Pegawai

|                                       |                                                      |
|---------------------------------------|------------------------------------------------------|
| Nama Lengkap                          | <input type="text" value="M.Yoga Ananda Satria"/>    |
| Email                                 | <input type="text" value="yogaananda205@gmail.com"/> |
| Nomor Telepon                         | <input type="text" value="085804709019"/>            |
| Tanggal Lahir                         | <input type="text" value="08/19/2005"/>              |
| Alamat                                | <input type="text" value="Dsn.Kebon Agung RT19 RV"/> |
| Tanggal Masuk                         | <input type="text" value="09/24/2025"/>              |
| Status                                | <input type="text" value="Aktif"/>                   |
| <input type="button" value="Update"/> |                                                      |



## Daftar Pegawai

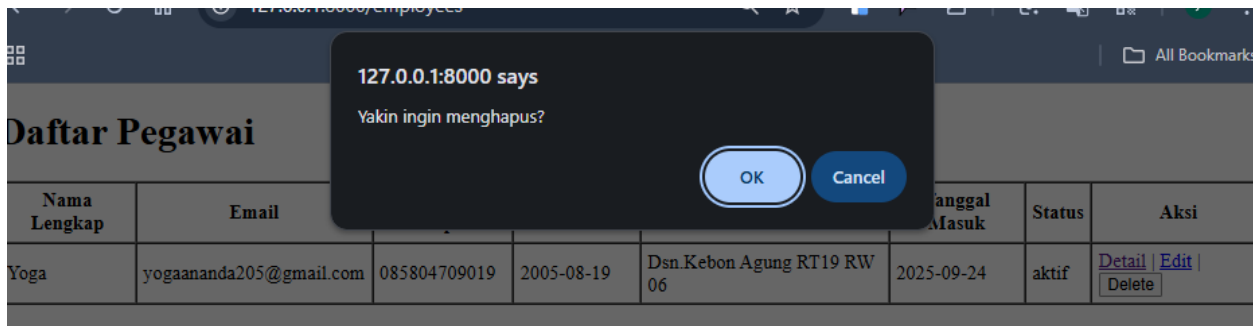
| Nama Lengkap | Email                   | Nomor Telepon | Tanggal Lahir | Alamat                     | Tanggal Masuk | Status | Aksi                                                                   |
|--------------|-------------------------|---------------|---------------|----------------------------|---------------|--------|------------------------------------------------------------------------|
| Yoga         | yogaananda205@gmail.com | 085804709019  | 2005-08-19    | Dsn.Kebon Agung RT19 RW 06 | 2025-09-24    | aktif  | <a href="#">Detail</a>   <a href="#">Edit</a>   <a href="#">Delete</a> |

## HAPUS DATA

1.Untuk menghapus data,langkah awal nya sama seperti percobaan yang sudah pernah saya buat sebelumnya yaitu masuk ke file **EmployeeController.php** dan masukan code berikut pada method **destroy**

```
...public function destroy(string $id): RedirectResponse
...{
...    $employee = Employee::find(id: $id);
...    $employee->delete();
...    return redirect()->route(route: 'employees.index');
...}
}
```

2. Uji coba dengan masuk pada route <http://127.0.0.1:8000/employees> kemudian klik button **Delete**, nanti akan keluar pop up “Yakin ingin menghapus?” setelah itu klik ok maka seluruh data yang sudah dimasukkan akan terhapus dan tersisa Cuma tabel saja seperti pada gambar di bawah ini:



## Daftar Pegawai

| Nama Lengkap | Email | Nomor Telepon | Tanggal Lahir | Alamat | Tanggal Masuk | Status | Aksi |
|--------------|-------|---------------|---------------|--------|---------------|--------|------|
|--------------|-------|---------------|---------------|--------|---------------|--------|------|

## D. Analisa

Berdasarkan percobaan yang dilakukan, pembuatan migration dan model pada Laravel berhasil menghasilkan tabel employees dengan kolom sesuai kebutuhan, seperti nama\_lengkap, email, dan status. Selanjutnya, implementasi CRUD melalui controller dan view dapat dijalankan dengan baik, mulai dari penambahan data baru ke dalam database, menampilkan data dalam bentuk tabel, menampilkan detail data tertentu, melakukan update pada data, hingga menghapus data yang sudah ada. Proses tersebut membuktikan bahwa integrasi antara migration, model, dan controller berjalan sesuai dengan fungsinya.

Selain itu, pengelolaan kode menggunakan Git pada branch dev membantu mahasiswa memahami pentingnya workflow version control dalam pengembangan perangkat lunak. Dengan melakukan commit dan push ke GitHub, mahasiswa dapat mempraktikkan alur kerja kolaboratif yang sering digunakan dalam industri. Dari hasil pengujian melalui browser, terlihat bahwa konfigurasi migration dan model sudah benar dan dapat menghasilkan tampilan sesuai dengan perintah artisan serta struktur MVC Laravel. Hal ini menunjukkan bahwa teori yang dipelajari telah berhasil diaplikasikan secara praktis dalam membangun aplikasi berbasis web.

## **E. Kesimpulan**

Dari hasil praktikum yang telah dilaksanakan, dapat disimpulkan bahwa penggunaan migration sangat membantu dalam pengelolaan struktur database karena lebih sistematis dan terdokumentasi. Model memudahkan interaksi dengan database menggunakan pendekatan berorientasi objek melalui Eloquent ORM, sehingga operasi query menjadi lebih ringkas dan efisien. Implementasi CRUD dapat berjalan dengan baik menggunakan kombinasi controller, model, dan view yang terintegrasi dalam pola MVC.

Selain itu, penerapan Git sebagai version control memberikan pengalaman nyata kepada mahasiswa dalam mengelola kode secara profesional, baik untuk pengembangan individu maupun kolaboratif. Secara keseluruhan, percobaan ini memberikan pemahaman yang lebih komprehensif tentang bagaimana Laravel dapat mengintegrasikan konsep database, pemrograman, dan manajemen proyek ke dalam satu ekosistem yang modern dan efisien.

## **F.Link Github**

<https://github.com/Rapprince29/Laravel-11>

